

Week 7: Lab

Solution

2025-10-23

RNA: WGCNA

Load packages

```
rm(list=ls())

pkgs <- c("tidyverse", "dplyr", "readr", "magrittr", "ggplot2", "readxl",
          "cowplot", "GGally", "knitr", "kableExtra", "data.table",
          "skimr", "flextable", "Metrics", "ggvenn", "devtools", "ape",
          "Biostrings", "DECIPHER", "edgeR", "WGCNA", "Spectra", "rpx",
          "mzR", "RCy3")
for (i in seq_along(pkgs)){
  pkg <- pkgs[[i]]
  tryCatch(if(pkg%in%rownames(installed.packages())){
    cat("Available:", pkg, "\n")
    library(pkg, character.only = TRUE)
    cat("Package loaded:", pkg, "\n")
  }
  else if(!(pkg%in%rownames(installed.packages()))){
    install.packages("BiocManager")
    BiocManager::install(pkg)
    library(pkg, character.only = TRUE)
    cat("Package loaded:", pkg, "\n")
  }
  else{
    cat("Not available:", pkg, ">> now installing...", "\n")
    install.packages(pkg, repos = "https://cloud.r-project.org")
    cat("Now available:", pkg, "\n")
  }
}
```

```

library(pkg, character.only = TRUE)
cat("Package loaded:", pkg, "\n")
}
)
}

```

Load data

```

## Set path
path <- "C:/Users/name/Desktop/GENET_5910"
#setwd(path) #Don't forget to remove # from setwd if you want R to recognize it

## Gene expression

ff1 <- sprintf("%s/LiverFemale3600.csv", path)

## Trait
ff2 <- sprintf("%s/ClinicalTraits.csv", path)

```

Read data

```

#####
## Gene expression
##
## Read the gene expression and only select columns containing individual
## information and gene expression levels and remove NA
#####

f_mice_gexp <- read_csv(ff1, col_select = c(1,9:143), show_col_types = F)|>
  drop_na()
f_mice_gexp[1:5, 1:5] ## View some of the data

```

```

# A tibble: 5 x 5
  substanceBXH    F2_2    F2_3    F2_14    F2_15
  <chr>         <dbl>   <dbl>   <dbl>   <dbl>
1 MMT00000044 -0.0181  0.0642  0.0000644 -0.058
2 MMT00000046 -0.0773 -0.0297  0.112     -0.0589
3 MMT00000051 -0.0226  0.0617 -0.129     0.0871

```

```
4 MMT000000080 -0.0487 0.0582 -0.0483 -0.0371
5 MMT000000102 0.176 -0.189 -0.065 -0.00846
```

```
#####
## Phenotype
##
## Read the trait and only select columns containing Mice IDs and variables
## of interest, select the female using filter(), and remove NA
#####

f_mice_y <- read_csv(ff2, col_select = c("Mice", "sex", "weight_g",
                                         "100xfat_weight", "total_fat",
                                         "Total_Cholesterol", "HDL_Cholesterol"),
                    show_col_types = F)|>
  filter(sex==2)|>
  drop_na()
f_mice_y[1:5, 1:5] ## View some of the data
```

```
# A tibble: 5 x 5
  Mice      sex weight_g `100xfat_weight` total_fat
  <chr> <dbl>   <dbl>         <dbl>     <dbl>
1 F2_290     2    36.9           13.0       4.79
2 F2_291     2    48.5           12.1       5.87
3 F2_296     2    38.8           12.7       4.92
4 F2_297     2    31.2           10.7       3.33
5 F2_298     2    33.4           12.0        4
```

```
# Match the IDs (column names) in the gene expression data with the
# IDs (first column) in the trait

common <- intersect(colnames(f_mice_gexp)[-1], f_mice_y$Mice)

## Select only IDs in both gene expression and trait

f_mice1 <- f_mice_gexp|>
  select(common)

f_mice_trait <- f_mice_y[f_mice_y$Mice%in%common,-2] #drop the sex in column 2
```

Quality Control

```
## Transpose the gene expression to ensure samples are in rows and
## genes are in columns. Then convert it to a data frame and
## rename the columns using gene IDs in the original f_mice_gexp object

f_mice <- as.data.frame(t(f_mice1), header=T)
names(f_mice) <- f_mice_gexp$substanceBXH

## Check for outliers in your gene
gsg.F <- goodSamplesGenes(f_mice)
```

Flagging genes and samples with too many missing values...
..step 1

```
summary(gsg.F)
```

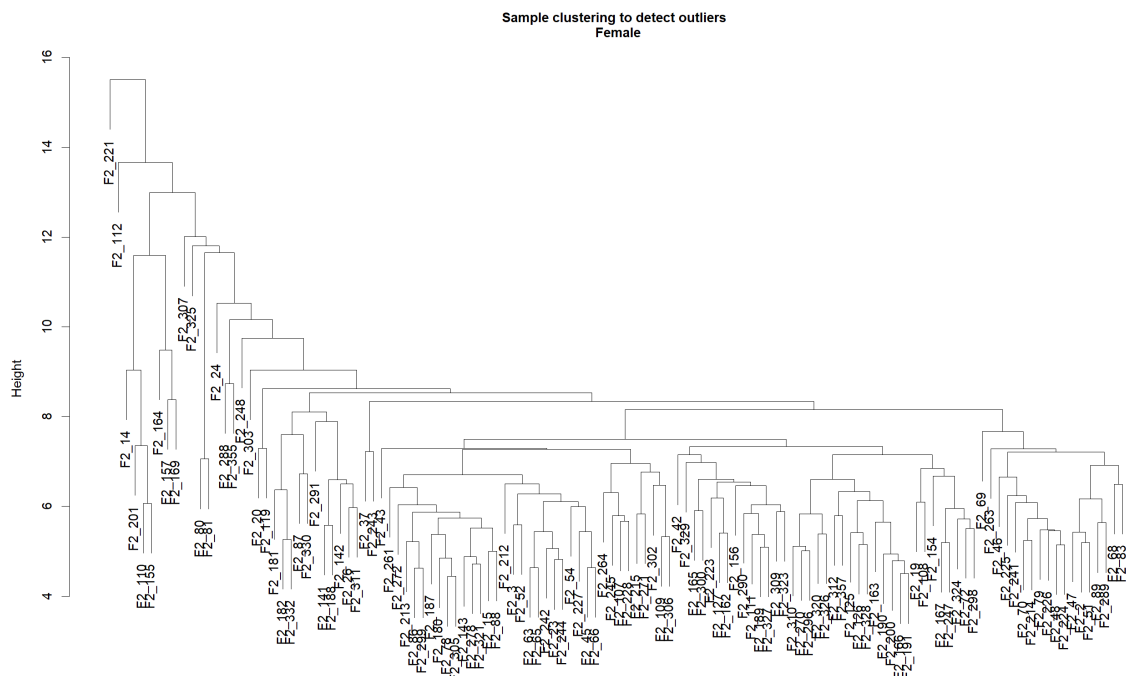
	Length	Class	Mode
goodGenes	2785	-none-	logical
goodSamples	124	-none-	logical
allOK	1	-none-	logical

```
gsg.F$allOK # A true value indicates no outlier
```

```
[1] TRUE
```

```
## Clustering samples based on average distance between clusters before merging
sampleTree.F <- hclust(dist(f_mice), method = "average")

#Plotting the cluster dendrogram
plot(sampleTree.F,
      main = "Sample clustering to detect outliers \n Female", sub="",
      xlab="", cex.lab = 1, cex.axis = 1, cex.main = 1)
```



Network Construction

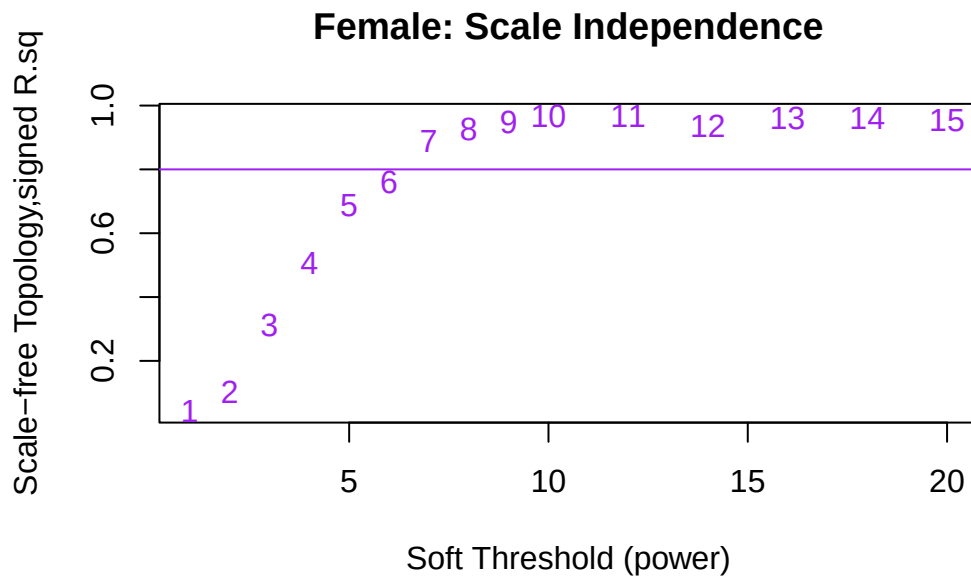
```
#####
## Calculate Soft power threshold for the network
##
## The 'pickSoftThreshold' helps in selecting the power needed to set the
## threshold for the similarity between expression levels of genes before
## computing the network adjacency matrix
##
## In the first plot, we set the  $R^2$  to 0.80 and any power (from 1-15) that
## exceeds that and also have the highest mean connectivity (in the second plot)
## will be chosen as the power
#####

spt.F <- pickSoftThreshold(f_mice)
```

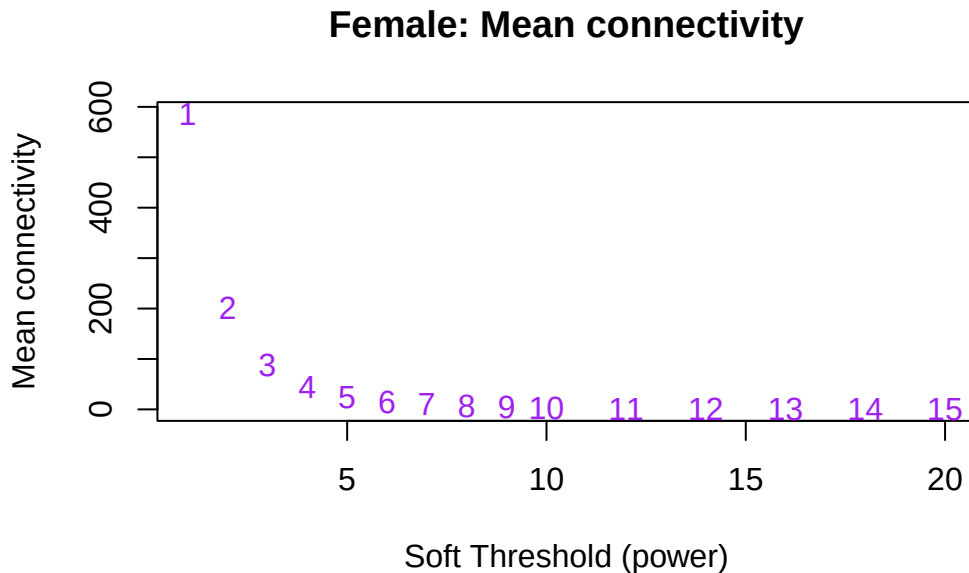
	Power	SFT.R.sq	slope	truncated.R.sq	mean.k.	median.k.	max.k.
1	1	0.0437	0.447	0.405	586.000	594.0000	926.0
2	2	0.1030	-0.533	0.853	201.000	198.0000	439.0
3	3	0.3140	-0.966	0.982	87.400	81.3000	242.0

4	4	0.5090	-1.330	0.991	44.200	38.1000	147.0
5	5	0.6870	-1.710	0.988	24.900	20.3000	96.7
6	6	0.7620	-1.760	0.973	15.200	11.6000	66.4
7	7	0.8890	-1.550	0.991	9.860	7.0400	47.1
8	8	0.9260	-1.710	0.990	6.750	4.4000	41.6
9	9	0.9480	-1.810	0.976	4.840	2.9000	38.0
10	10	0.9680	-1.790	0.974	3.600	1.9600	35.2
11	12	0.9680	-1.720	0.959	2.180	0.9500	30.8
12	14	0.9370	-1.650	0.919	1.460	0.4850	27.5
13	16	0.9610	-1.520	0.953	1.050	0.2580	24.8
14	18	0.9630	-1.450	0.957	0.797	0.1390	22.6
15	20	0.9550	-1.390	0.946	0.631	0.0791	20.7

```
plot(spt.F$fitIndices[,1], spt.F$fitIndices[,2], type="n", ####
     main = "Female: Scale Independence",
     xlab = "Soft Threshold (power)",
     ylab = "Scale-free Topology,signed R.sq")
text(spt.F$fitIndices[,1], spt.F$fitIndices[,2], col="purple")
abline(h=0.80, col="purple")
```



```
plot(spt.F$fitIndices[,1], spt.F$fitIndices[,5], type="n",
     main = "Female: Mean connectivity",
     xlab = "Soft Threshold (power)", ylab = "Mean connectivity")
text(spt.F$fitIndices[,1], spt.F$fitIndices[,5], col="purple")
```



```
## Calling adjacency
## Although I gave you 6, 7 actually exceeds the 0.80 R^2 and highest mean among
## power that exceeds 0.80
sp.F <- 7

adj.F <- adjacency(f_mice, power = sp.F)
```

Module Construction

```
## Defining dissimilarity in the adjacency matrix

TOM.F <- TOMsimilarity(adj.F) ## First obtain the similarity

..connectivity..
```

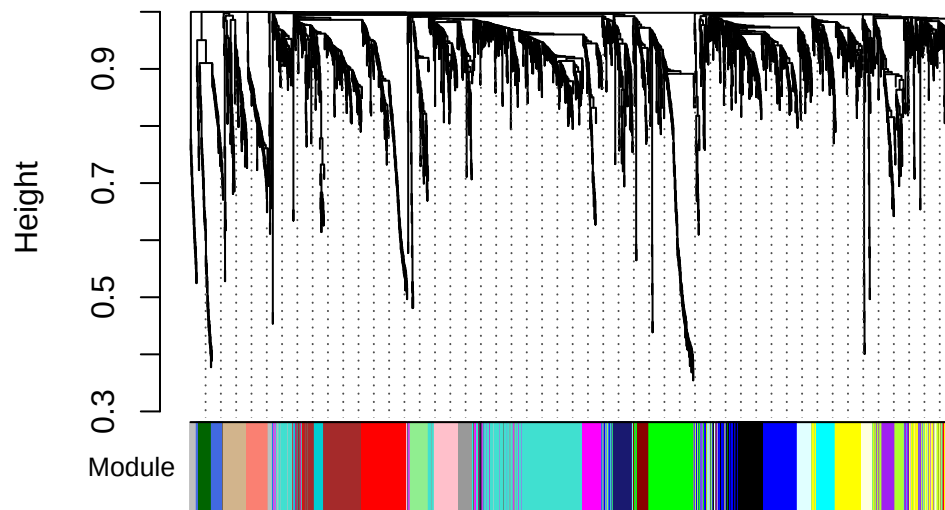
```
..matrix multiplication (system BLAS)..  
..normalization..  
..done.
```

```
TOM.dissim.F <- 1-TOM.F ## Then 1-similarity give dissimilarity  
  
## Hierarchical clustering analysis to visualize the dissimilarity clusters  
  
geneT.F <- hclust(as.dist(TOM.dissim.F), method = "average")  
  
## Set module size for the gene dendrogram and the minimum gene cluster size  
  
mod.F <- cutreeDynamic(dendro=geneT.F, distM = TOM.dissim.F, deepSplit = 2,  
                      pamRespectsDendro = FALSE, minClusterSize = 30)
```

```
..cutHeight not given, setting it to 0.996 ==> 99% of the (truncated) height range in de  
..done.
```

```
## Module color  
## Assign color to each gene module  
  
mod.col.F <- labels2colors(mod.F)  
  
## Plot the gene dendrogram with module color  
plotDendroAndColors(geneT.F, mod.col.F, "Module", dendroLabels=FALSE, hang=0.03,  
                   addGuide=TRUE, guideHang=0.05,  
                   main="Female: Gene dendrogram and module colors")
```


Female: Gene dendrogram and module colors



```
## Module Eigengene Identification

MElist.F <- moduleEigengenes(f_mice, colors=mod.col.F)

MEs.F <- MElist.F$eigengenes #Retrieve the eigengene list
```

Module merging

```
#####
## To synthesize module detection and for computational efficiency, we want to
## merge modules with highly correlated eigengenes
#####

## Eigengene dissimilarity

ME.dissim.F <- 1-cor(MElist.F$eigengenes, use="complete")

## Clustering eigengenes dissimilarity
## Here the clustering reveals gene modules that can be merged together based on
## their dissimilarity
```

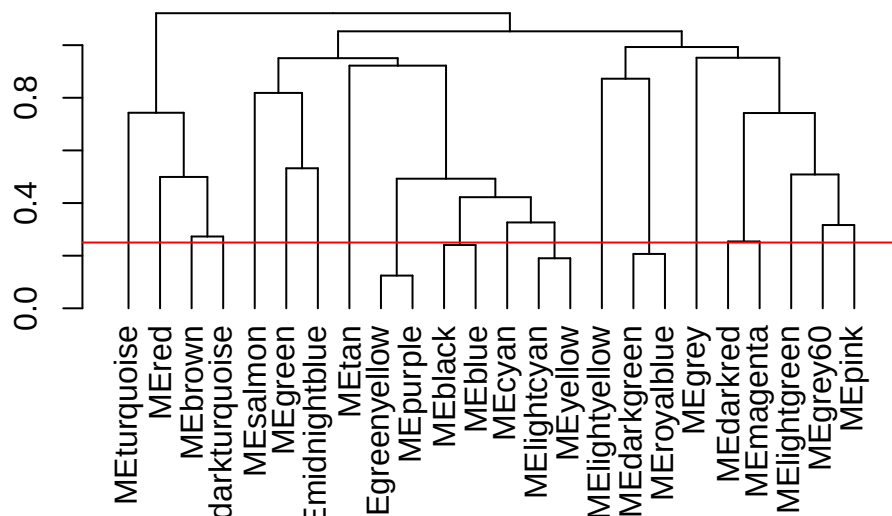
```

METree.F <- hclust(as.dist(ME.dissim.F), method="average")

## plot tree showing clustering of eigengenes with a cut-off of 0.25
plot(as.dendrogram(METree.F), main = "Female: Cluster of module eigengenes")
abline(h=.25, col="red")

```

Female: Cluster of module eigengenes



```

## Merge module
## Here gene modules in a hierarchical cluster below the 0.25 height will be merged
## together

#Threshold for merging modules set at .25
merge.F <- mergeCloseModules(f_mice, mod.col.F, cutHeight=.25)

```

mergeCloseModules: Merging modules whose distance is less than 0.25
Calculating new MEs...

```

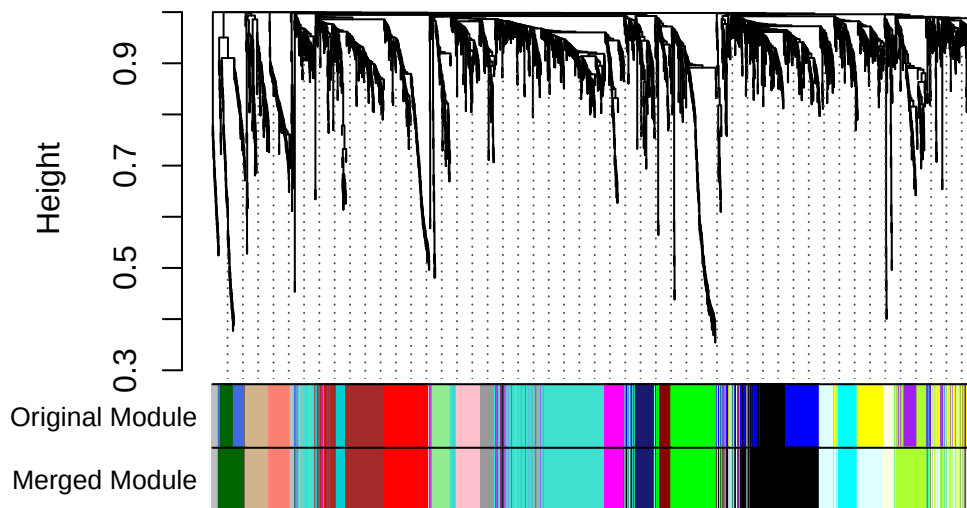
merged.col.F=merge.F$colors #merge colors and assign color to each module
mergedMEc.F <- merge.F$newMEs #eigengenes of the new merged modules

## Plot the gene dendrogram with module color
plotDendroAndColors(geneT.F, cbind(mod.col.F,merged.col.F),

```

```
c("Original Module","Merged Module"),
dendroLabels=FALSE, hang=0.03, addGuide=TRUE,
guideHang=0.05,
main="Female: Gene dendrogram & module colors for original &
merged modules")
```

Female: Gene dendrogram & module colors for original & merged modules



Trait matching

Module-trait association

```
nGenes_f <- ncol(f_mice) ## obtain the number of genes

nSamples_f <- nrow(f_mice) ## obtain the number of samples

y_f2 <- f_mice_trait|>
  column_to_rownames(var="Mice") ## Change sample IDs from column to rownames

##subset merged eigengenes using the IDs in the trait. Note the IDs are
## the rownames
mergedMEc.F2 <- mergedMEc.F[rownames(y_f2),]
```

```

## Obtain the correlation and the correlation p-values between the merged
## eigengenes and the trait

mod.trait.cor_f <- cor(mergedMEc.F2, y_f2, use="p")

mod.trait.pv_f <- corPvalueStudent(mod.trait.cor_f, nSamples_f)

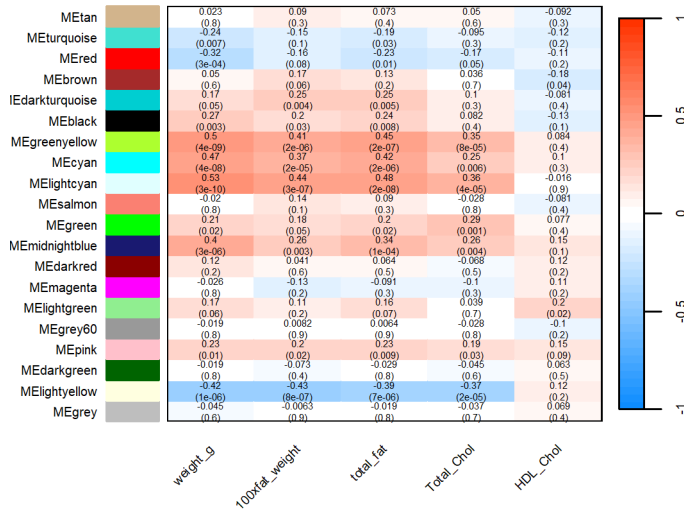
## Plot the Module-trait heatmap
## Specify the how you want the correlation coefficient values and p-values to
## be displayed

textMatrix_f <- paste(signif(mod.trait.cor_f, 2),
                        "\n(", signif(mod.trait.pv_f, 1), ")", sep = "");
dim(textMatrix_f) = dim(mod.trait.cor_f)

labeledHeatmap(Matrix = mod.trait.cor_f,
                xLabels = names(y_f2),
                yLabels = names(mergedMEc.F2),
                ySymbols = names(mergedMEc.F2),
                colorLabels = F,
                colors = blueWhiteRed(50),
                textMatrix = textMatrix_f,
                setStdMargins = F,
                cex.text = 0.3,
                zlim = c(-1,1),
                main=paste("Female: Module-trait relationships"),
                cex.lab = 0.4
                )

```

Female: Module-trait relationships



Visualize the top 4 module in each trait using a Venn diagram

```
## Here, the top 4 modules with the highest correlation values and pvalues in
## each trait are selected to see the number of genes within each module
##
## You can see that green, magenta, and midnightblue modules have high
## correlation with the selected traits
```

```
weight <- list(MLCyan=names(f_mice[merged.col.F=="lightcyan"]),
               MGYellow=names(f_mice[merged.col.F=="greenyellow"]),
               MCyan=names(f_mice[merged.col.F=="cyan"]),
               MMNBlue=names(f_mice[merged.col.F=="midnightblue"]))

T_fat <- list(MLCyan=names(f_mice[merged.col.F=="lightcyan"]),
              MGYellow=names(f_mice[merged.col.F=="greenyellow"]),
              MCyan=names(f_mice[merged.col.F=="cyan"]),
              MMNBlue=names(f_mice[merged.col.F=="midnightblue"]))
```

```

x_fat_wght <- list(MLCyan=names(f_mice[merged.col.F=="lightcyan"]),
  MGYellow=names(f_mice[merged.col.F=="greenyellow"]),
  MCyan=names(f_mice[merged.col.F=="cyan"]),
  MMNBlue=names(f_mice[merged.col.F=="midnightblue"]))

T_chol <- list(MLCyan=names(f_mice[merged.col.F=="lightcyan"]),
  MGYellow=names(f_mice[merged.col.F=="greenyellow"]),
  MGreen=names(f_mice[merged.col.F=="green"]),
  MMNBlue=names(f_mice[merged.col.F=="midnightblue"]))

F1 <- ggvenn(weight,
  stroke_size = 0.5, set_name_size = 4, text_size = 3.3, show_percentage = FALSE)

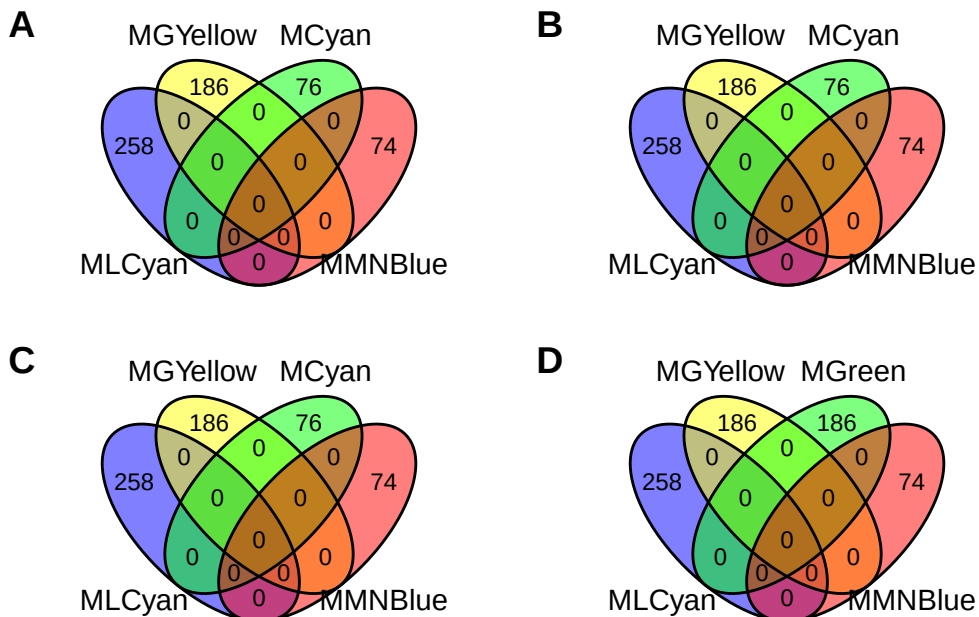
F2 <- ggvenn(T_fat,
  stroke_size = 0.5, set_name_size = 4, text_size = 3.3, show_percentage = FALSE)

F3 <- ggvenn(x_fat_wght,
  stroke_size = 0.5, set_name_size = 4, text_size = 3.3, show_percentage = FALSE)

F4 <- ggvenn(T_chol,
  stroke_size = 0.5, set_name_size = 4, text_size = 3.3, show_percentage = FALSE)

plot_grid(F1,F2,F3,F4, scale = 1, labels = c("A","B", "C", "D"))

```



Export Networks to Cytoscape

```
#####  
##### DO NOT RUN #####  
#####  
## You'll need to install cytoscape if you want to run this script  
## After installing cytoscape, ensure you create a folder in the directory  
## where you data is located. Name the folder Cytoscape.  
## Make sure the set path to the directory as stated in the Load data chunk  
  
# for (i in 1:length(merge.F$oldMEs)){  
#   modules = c(substring(names(merge.F$oldMEs)[i], 3));  
#   genes = colnames(f_mice)  
#   inModule = is.finite(match(mod.col.F,modules))  
#   modGenes = genes[inModule]  
#   modTOM=TOM.F[inModule,inModule]  
#   dimnames(modTOM)=list(modGenes,modGenes)  
#   cyt = exportNetworkToCytoscape(modTOM,  
#                                   edgeFile =  
#                                   paste("Cytoscape/orign_CytoscapeInput-edges-",  
#                                       paste(modules, collapse="-"), ".txt",  
#                                       sep=""),  
#                                   nodeFile =  
#                                   paste("Cytoscape/orign_CytoscapeInput-nodes-",  
#                                       paste(modules, collapse="-"), ".txt",  
#                                       sep=""),  
#                                   weighted = TRUE, threshold = -1,  
#                                   nodeNames = modGenes,  
#                                   nodeAttr = mod.col.F[inModule]);  
# }  
# # Export the gene list of new modules  
# for (i in 1:length(merge.F$newMEs)){  
#   modules = c(substring(names(merge.F$newMEs)[i], 3));  
#   genes = colnames(f_mice)  
#   inModule = is.finite(match(mod.col.F,modules))  
#   modGenes = genes[inModule]  
#   modTOM=TOM.F[inModule,inModule]  
#   dimnames(modTOM)=list(modGenes,modGenes)  
#   cyt = exportNetworkToCytoscape(modTOM,  
#                                   edgeFile =  
#                                   paste("Cytoscape/merge_CytoscapeInput-edges-",  
#                                       paste(modules, collapse="-"), ".txt",  
#                                       sep=""),  
#                                   nodeFile =  
#                                   paste("Cytoscape/merge_CytoscapeInput-nodes-",  
#                                       paste(modules, collapse="-"), ".txt",  
#                                       sep=""),  
#                                   weighted = TRUE, threshold = -1,  
#                                   nodeNames = modGenes,  
#                                   nodeAttr = mod.col.F[inModule]);  
# }
```

```

#                                     sep=""),
#                                     nodeFile =
#                                     paste("Cytoscape/merge_CytoscapeInput-nodes-",
#                                     paste(modules, collapse="-"), ".txt",
#                                     sep=""),
#                                     weighted = TRUE, threshold = -1,
#                                     nodeName = modGenes,
#                                     nodeAttr =mod.col.F[inModule]);
# }

```

Visualize networks in cytoscape by module colors

```

#####
##### DO NOT RUN #####
#####

# edge <- read_delim("Cytoscape/merge_CytoscapeInput-edges-green.txt",
#                   show_col_types = F)
# colnames(edge)
# colnames(edge) <- c("source", "target","weight","direction","fromAltName",
#                   "toAltName")
#
# node <- read_delim("Cytoscape/merge_CytoscapeInput-nodes-green.txt",
#                   show_col_types = F)
# colnames(node)
# colnames(node) <- c("id","altName","node_attributes")
#
# createNetworkFromDataFrames(node,edge[1:50,], title="my first network",
#                             collection="DataFrame Example")

```

```

## This is for style your network

# setVisualStyle('Marquee')
#
# # set up my own style
# style.name = "myStyle"
# defaults <- list(NODE_SHAPE="diamond",
#                 NODE_SIZE=30,
#                 EDGE_TRANSPARENCY=120,
#                 NODE_LABEL_POSITION="W,E,c,0.00,0.00")

```


MS-Proteomics Data Analysis

```
#####  
## Here we are using the 'rpx' package to retrieve data from the ProteomicsXchange  
## dataset using the PX project identifier "PXD057454"  
## Go here to find more: https://proteomecentral.proteomexchange.org/  
## I ensure the dataset has .mzML extension before selecting it.  
## The .mzML extension is one of the mass spectrometry data file extension that  
## the Spectra packages accepts  
#####  
  
pxd <- PXDataset("PXD057454") #Retrieve the data
```

Loading PXD057454 from cache.

```
## Check the files in the data. Note it prints the first 10 files. To see more,  
## e.g 20, use pxfiles(pxd, n=20).  
  
pxfiles(pxd)
```

Project PXD057454 files (15):

```
[remote] 2025-02-23-decoys-contam-WH8109_UP000002780_S-TIP37_UP00255828_mod.fasta.fas  
[remote] File_Description  
[remote] Nt-DiMet_newDB.zip  
[remote] Nt-MetP_newDB.zip  
[local] Seq84734_88min_480Ex1.mzML  
[remote] Seq84734_88min_480Ex1.raw  
[remote] Seq84735_88min_480Ex1.mzML  
[remote] Seq84735_88min_480Ex1.raw  
[remote] Seq84736_88min_480Ex1.mzML  
[remote] Seq84736_88min_480Ex1.raw  
...
```

```
fl <- "Seq84734_88min_480Ex1.mzML" #select the .mzML file  
mz11 <- pxget(pxd,fl) #Download the .mzML file
```

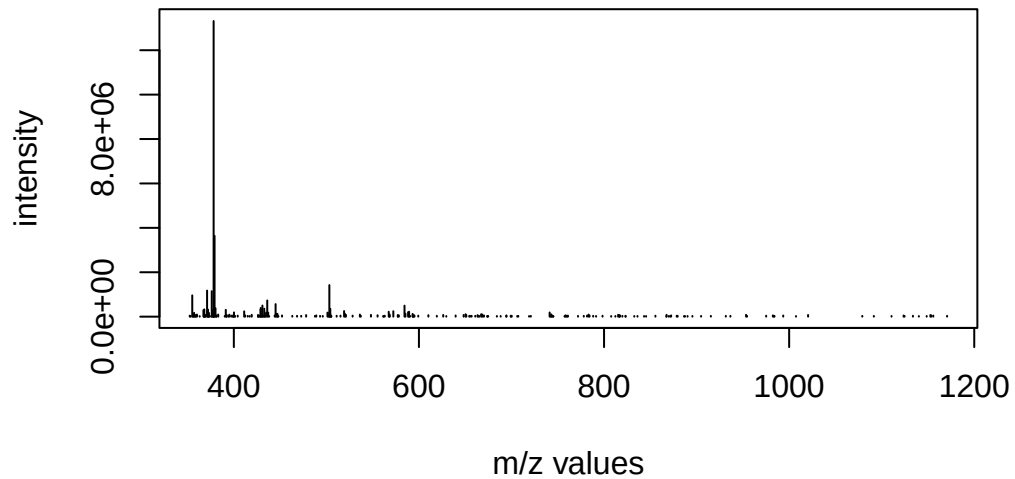
Loading Seq84734_88min_480Ex1.mzML from cache.

```
## Access the raw data and obtain the spectra information, mass-charge values,
## peaks and peak intensity, etc

spt <- Spectra(mz11)

## Plot intensity against mass-charge values

plot(mz(spt)[[1]], intensity(spt)[[1]], type="h", xlab="m/z values",
      ylab="intensity")
```



```
table(msLevel(spt)) #No of MS levels and No of spectra per MS levels
```

```
      1      2
5277 89061
```

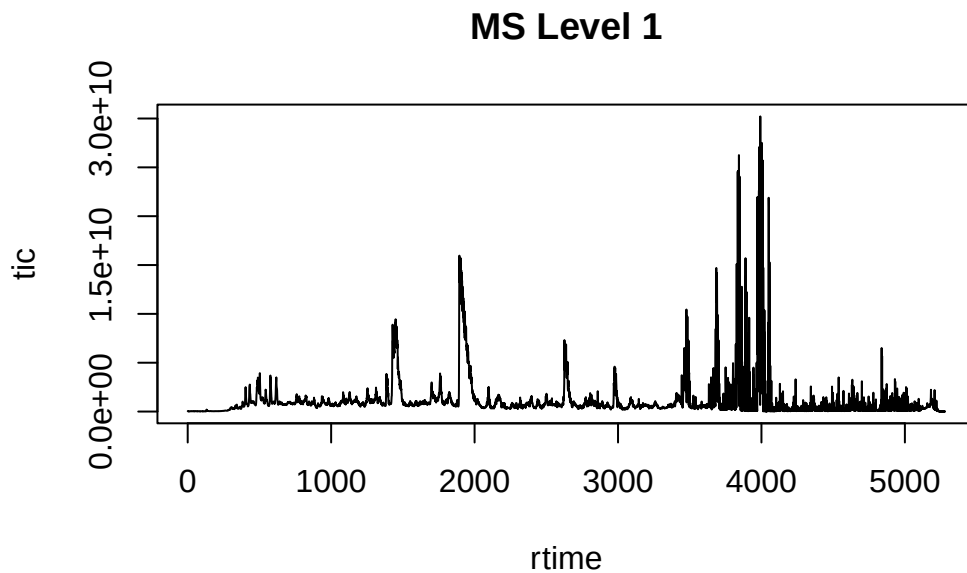
```
range(rtime(spt)) #Range of retention time
```

```
[1]      0.103051 5280.191442
```

```
spt1 <- filterMsLevel(spt,1) #Filter MS level 1

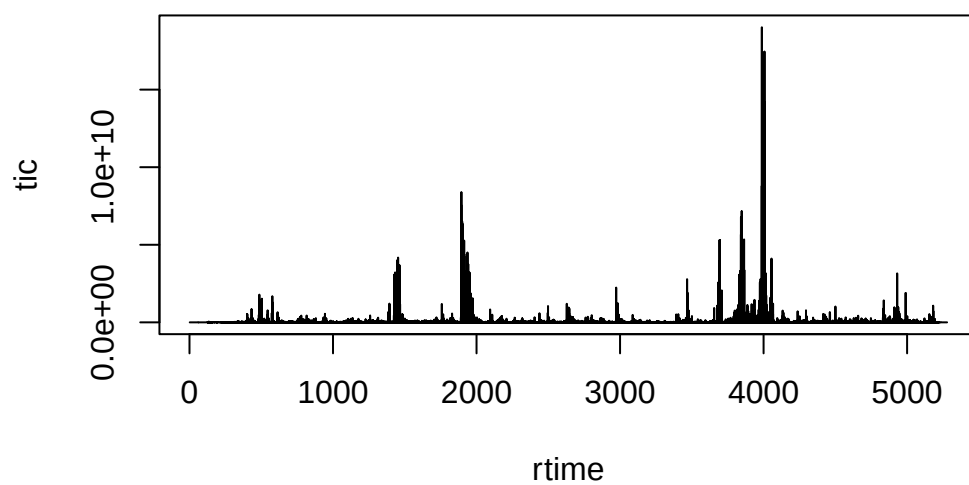
spt2 <- filterMsLevel(spt,2) #Filter MS level 2

## Plot total ion current against retention time in MS level 1
plot(rtime(spt1), tic(spt1), type="l", xlab="rtime", ylab="tic", main="MS Level 1")
```



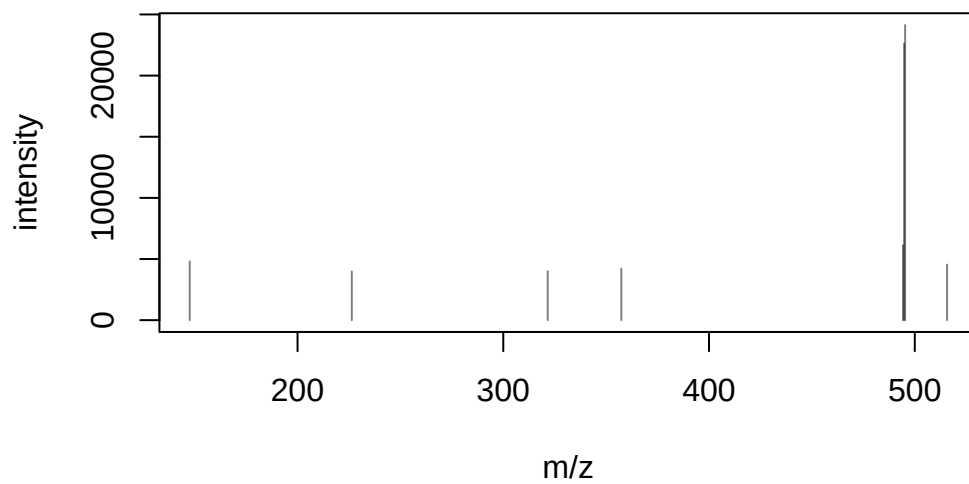
```
## Plot total ion current against retention time in MS level 2
plot(rtime(spt2), tic(spt2), type="l", xlab="rtime", ylab="tic", main="MS Level 2")
```

MS Level 2



```
## Plot intensity against mass-charge values for the first 2000 spectra  
plotSpectra(spt[2000])
```

MS2 RT: 231.3



```
## Highest precursor intensity for MS level 1
mxm_1 <- max(precursorIntensity(spt1))
mxm_1
```

```
[1] NA
```

```
## Location of highest precursor intensity
wch_1 <- which(precursorIntensity(spt1)==mxm_1)
wch_1
```

```
integer(0)
```

```
## Index of the MS level 1 spectrum with the highest precursor intensity
scanIndex(spt1)[wch_1]
```

```
integer(0)
```

```
## Highest base peak intensity
mxm_1b <- max(spt1$basePeakIntensity)
mxm_1b
```

```
[1] 7977000960
```

```
## Location of highest base peak intensity
which(spt1$basePeakIntensity==mxm_1b)
```

```
[1] 2033
```

```
## Highest precursor intensity for MS level 2
mxm_2 <- max(precursorIntensity(spt2))
mxm_2
```

```
[1] 16136117287
```

```
## Location of highest precursor intensity
wch_2 <- which(precursorIntensity(spt2)==mxm_2)
wch_2
```

```
[1] 68140
```

```
## Index of the MS2 spectrum with the highest precursor intensity  
scanIndex(spt2)[wch_2]
```

```
[1] 72115
```

```
## Highest base peak intensity  
mxm_2b <- max(spt2$basePeakIntensity)  
mxm_2b
```

```
[1] 2874288384
```

```
## Location of highest base peak intensity  
which(spt2$basePeakIntensity==mxm_2b)
```

```
[1] 31032
```